Relatório técnico - Gerar_base_geocodificada.py

Aluno: Jackson David Rodrigues Penaforte

Curso Superior de Tecnologia em Logística Instituição: Faculdade de Tecnologia de São Paulo — FATEC Jessen Vidal

Projeto: LOGTEAM 3 / IPEM-SP
Arquivo analisado: gerar_base_geocodificada.py
Quantidade: 512 linhas
Objetivo: gerar uma base de fiscalização enriquecida com identificadores, endereços padronizados, coordenadas geográficas, links de rota, distâncias e tempos de deslocamento.

1. Resumo executivo

O programa implementa um pipeline ETL geográfico. Ele lê data v1.csv, mantém a rastreabilidade da linha original, padroniza endereços, transforma registros detalhados em visitas únicas por equipe/dia/estabelecimento, elimina fiscais com menos de seis visitas, geocodifica endereços usando Nominatim, calcula deslocamentos usando Haversine e, quando possível, OSRM, e exporta o resultado para Excel e CSV. O arquivo Excel é criado com uma aba única, cabeçalho formatado, filtros e primeira linha congelada.

2. Fluxo completo de execução

1 Monta caminhos, parâmetros, sede, limites e nomes das colunas.
2 Lê a base CSV com codificação latin1.
3 Cria rastreabilidade, endereço normalizado, IDs de equipe/fiscalização e datas.
4 Consolida visitas e filtra fiscais com menos de seis visitas.
5 Calcula o total de visitas por equipe e dia.
6 Geocodifica endereços únicos em cascata e reutiliza cache.
7 Localiza a sede e calcula trechos na ordem temporal.
8 Tenta substituir estimativas por rotas OSRM para cada equipe-dia.
9 Mescla os resultados na base detalhada, arredonda números e seleciona colunas.
10 Exporta Base_Geocodificada.xlsx e, se habilitado, Base_Geocodificada.csv.

3. Entradas, saídas e dependências

- **Entrada:** data v1.csv, localizado em PASTA; são esperadas colunas como ENDERECO_COMPLETO, IDFISCAL, IDMOTORISTA, SERVIÇO, DT_HORA_VERIF, DATAVERIFICACAO, COD_PROPR e os campos exibidos em COLUNAS_SAIDA.
- **Saídas:** Base_Geocodificada.xlsx e, com EXPORTAR_CSV=True, Base_Geocodificada.csv.

- **Dependências Python:** json, os, re, time, urllib.parse, numpy, pandas, requests e, na exportação, openpyxl.
- **Serviços externos:** Nominatim/OpenStreetMap para geocodificação; OSRM para rotas; Google Maps apenas como link navegável, não como serviço consultado pelo programa.

4. Regras de negócio implementadas

- Visita = combinação de equipe, dia e estabelecimento (COD_PROPR).
- Fiscal com menos de MIN_VISITAS_FISCAL visitas é removido da base.
- ID_EQUIPE segue EQP-IDFISCAL-IDMOTORISTA.
- Tipo de fiscalização é numerado pela ordem alfabética dos serviços.
- Registro de fiscalização preserva a posição original no CSV.
- Primeiro trecho do dia parte da sede; os seguintes partem da visita anterior.
- Link de retorno e retorno numérico são associados à última visita do dia.
- Geocodificação usa maior precisão possível e cai para município quando necessário.
- Falhas de OSRM mantêm a estimativa baseada em linha reta, fator 1,30 e velocidade de 40 km/h.

5. Pontos de atenção técnicos

11 **Ordenação de semanas/datas:** o script não cria uma dimensão calendário; análises semanais no Power BI devem ser feitas a partir de DT_HORA_VERIF/DATAVERIFICACAO ou de uma tabela calendário relacionada.

12 **Semana que cruza mês:** não existe regra de semana no script; essa lógica precisa ser criada no Power BI ou em etapa adicional.

13 **Nominatim/OSRM públicos:** há pausas e cache, mas os serviços públicos podem impor limites, alterar disponibilidade ou exigir política de uso.

14 **Precisão:** MUNICIPIO é um fallback de baixa precisão; NAO_ENCONTRADO deixa coordenadas nulas.

15 **Filtro por fiscal:** a eliminação acontece antes da geocodificação e dos cálculos de rota, portanto os descartados não aparecem na saída.

16 **Data inválida:** quando DT_HORA_VERIF falha, o código usa DATAVERIFICACAO; se ambos falharem, DIA pode ficar nulo e afetar agrupamentos.

17 **Cache:** falhas de rede não são cacheadas, mas resultados sem ocorrência podem ser cacheados pelo Nominatim como None, dependendo do caminho de execução.

18 **Sede:** a coordenada aproximada deve ser revisada se a sede real mudar; a configuração manual é o mecanismo mais seguro.

6. Relatório linha por linha

A tabela abaixo transcreve cada linha do arquivo e explica sua função. Linhas em branco e comentários também são registradas para manter a correspondência exata com o código original.

Linha	Código	Explicação
1	"""	Abre a docstring de documentação do módulo.
2	Base geocodificada por visita | LOGTEAM 3 | IPEM-SP	Identifica o produto/base: base geocodificada por visita, do projeto LOGTEAM 3 / IPEM-SP.
3	``	Informa o arquivo de entrada esperado: data v1.csv.
4	Entrada : data v1.csv (base original, mesma pasta do Drive)	Informa a saída Excel e sua regra: uma única aba chamada Base_Geocodificada, com colunas em ordem definida.
5	Saída : Base_Geocodificada.xlsx -> UMA aba ("Base_Geocodificada") com as colunas pedidas, na ordem pedida	Informa a saída CSV e que ela reproduz o mesmo conteúdo quando EXPORTAR_CSV estiver ativo.
6	Base_Geocodificada.csv -> mesmo conteúdo em .csv (EXPORTAR_CSV)	Linha em branco/separação visual dentro da documentação.
7	``	Linha em branco/separação visual dentro da documentação.
8	REGRAS	Inicia a seção de regras de negócio.
9	* Fiscais com menos de MIN_VISITAS_FISCAL (6) visitas no período são descartados.	Define o filtro mínimo: fiscais com menos de seis visitas no período são descartados; visita é estabelecimento visitado por fiscal em um dia.
10	Visita = um estabelecimento (COD_PROPR) visitado por um fiscal em um dia.	Continua a definição operacional de visita.

Linha	Código	Explicação
11	* ENDERECO_PADRONIZADO = endereço sem espaços duplos, em maiúsculas, com DOUTOR->DR e AVENIDA->AV	Define a padronização do endereço: remove espaços duplicados, coloca em maiúsculas e substitui DOUTOR por DR e AVENIDA por AV.
12	(mesma padronização do ETL anterior, conferida linha a linha).	Registra que a padronização deve manter compatibilidade com o ETL anterior.
13	* ID_EQUIPE = EQP-<IDFISCAL>-<IDMOTORISTA> (0 quando não há motorista).	Define o formato do identificador de equipe e o uso de zero quando não há motorista.
14	ID_TIPO_FISCALIZACAO = FISC-001, 002... pela ordem alfabética do SERVIÇO.	Define a codificação de tipo de fiscalização pela ordem alfabética do serviço.
15	ID_REGISTRO_FISCALIZACAO = FISC-REC-<nº da linha no data v1.csv> (mantido mesmo após descartes).	Define o identificador do registro com base na linha original do CSV, preservando-o mesmo após filtros.
16	* TOTAL_VISITAS_DIA = estabelecimentos distintos visitados pela equipe no dia.	Define TOTAL_VISITAS_DIA como quantidade de estabelecimentos distintos visitados pela equipe no dia.
17	* Trecho = deslocamento até a visita, a partir da visita anterior da equipe no dia (ordem do	Define o trecho como o deslocamento da visita anterior da equipe no dia, ou da central na primeira visita.
18	horário) ou da central, na 1ª visita. O LINK_RETORNO_SEDE aparece nas linhas da última visita.	Define que o link de retorno à sede fica associado à última visita do dia.
19	* LATITUDE/LONGITUDE/PRECISAO_GEO: geocodificação em cascata (ENDERECO > RUA > CEP > BAIRRO > MUNICIPIO).	Define a cascata de geocodificação: endereço completo, rua, CEP, bairro e município.
20	* DISTANCIA_DIRETA_KM = linha reta; DISTANCIA_RODOVIARIA_ESTIMADA_KM e TEMPO_DESLOCAMENTO_MIN = rota por	Define a distância em linha reta e a distância/tempo rodoviários usando OSRM por equipe-dia.

Linha	Código	Explicação
21	estrada (OSRM, uma consulta por equipe-dia); onde não houver rota, linha reta x fator e velocidade média.	Define o fallback: quando não houver rota, usa distância em linha reta multiplicada por fator e velocidade média.
22	"""	Fecha a docstring do módulo.
23	import json	Importa json para leitura e gravação dos caches.
24	import os	Importa os para manipulação de caminhos e verificação de arquivos.
25	import re	Importa re para expressões regulares usadas em endereços e limpeza.
26	import time	Importa time para pausas entre requisições e retentativas.
27	from urllib.parse import quote	Importa quote para codificar endereços nos links do Google Maps.
28	``	Linha em branco usada apenas para separação visual.
29	import numpy as np	Importa NumPy para operações numéricas, arrays, NaN e cálculos vetorizados.
30	import pandas as pd	Importa pandas para leitura, transformação, agrupamento e exportação tabular.
31	import requests	Importa requests para chamadas HTTP ao Nominatim e ao OSRM.
32	``	Linha em branco usada apenas para separação visual.
33	# ---------- CONFIGURAÇÃO (editável) ----------	Inicia a seção de configuração editável.
34	PASTA = "/content/drive/MyDrive/LOG3/API3/LOGTEAM 3"	Define a pasta do Google Drive/Colab onde estão entrada, caches e saídas.

Linha	Código	Explicação
35	ARQ_ENTRADA = os.path.join(PASTA, "data v1.csv")	Monta o caminho completo do CSV de entrada.
36	NOME_SAIDA = "Base_Geocodificada"	Define o nome-base dos arquivos de saída.
37	EXPORTAR_CSV = True	Ativa a geração do CSV adicional.
38	ARQ_CACHE_GEO = os.path.join(PASTA, "cache_geocodificacao.json") # o mesmo cache do programa anterior	Define o arquivo de cache de geocodificação, compartilhado com o programa anterior.
39	ARQ_CACHE_ROTAS = os.path.join(PASTA, "cache_rotas_osrm.json")	Define o arquivo de cache das rotas OSRM.
40	``	Linha em branco usada apenas para separação visual.
41	MIN_VISITAS_FISCAL = 6 # fiscais com menos visitas que isso são descartados	Define o limite mínimo de visitas por fiscal.
42	``	Linha em branco usada apenas para separação visual.
43	USER_AGENT = "LOGTEAM3-FATEC-IPEM/1.0 (contato: jackson.penaforte@aluno.cps.sp.gov.br)"	Define o User-Agent enviado às APIs, incluindo identificação e contato.
44	URL_NOMINATIM = "https://nominatim.openstreetmap.org/search"	Define o endpoint de busca do Nominatim/OpenStreetMap.
45	PAUSA_S = 1.1	Define a pausa padrão de 1,1 segundo entre consultas de geocodificação.
46	RAIO_MAX_KM = 40	Define o raio máximo aceitável entre ponto encontrado e centro do município.
47	SALVAR_A_CADA = 50	Define que o cache de geocodificação é salvo a cada 50 consultas novas.
48	LIMITE_TESTE = None # ex.: 100 para testar com poucos endereços	Permite limitar a quantidade de endereços para teste; None significa processar todos.

Linha	Código	Explicação
49	``	Linha em branco usada apenas para separação visual.
50	SEDE_MANUAL = None # ex.: (-23.1806, -45.8829)	Permite informar manualmente as coordenadas da sede; None ativa a busca automática.
51	SEDE_APROX = (-23.1791, -45.8872)	Define coordenada aproximada de fallback para o centro de São José dos Campos.
52	SEDE_ENDERECO = "Rua Sebastião Hummel, 759 - Centro, São José dos Campos - SP (CEP 12210-200)"	Define o endereço textual da sede.
53	SEDE_TEXTO_LINK = "R. SEBASTIÃO HUMEL, 759 - CENTRO, SÃO JOSÉ DOS CAMPOS - SP, 12210-200" # origem/destino nos links	Define o texto usado como origem/destino nos links do Google Maps.
54	``	Linha em branco usada apenas para separação visual.
55	USAR_OSRM = True	Ativa o uso do OSRM para obter rotas rodoviárias.
56	URL_OSRM = "https://router.project-osrm.org/route/v1/driving/"	Define o endpoint público do OSRM no perfil driving.
57	OSRM_PAUSA_S = 1.0	Define pausa de um segundo entre consultas de rota.
58	FATOR_CIRCUIDADE = 1.30 # reserva (linha reta): estrada ~30% maior	Define o fator de circuidade de 1,30 para estimar estrada quando só há linha reta.
59	VELOCIDADE_KMH = 40.0 # reserva (linha reta): velocidade média	Define velocidade média de 40 km/h para converter distância estimada em tempo.
60	INCLUIR_COLUNAS_RETORNO = False # True acrescenta DISTANCIA/TEMPO do retorno à central nas linhas da última visita	Desativa, por padrão, colunas numéricas explícitas de retorno à sede.
61	``	Linha em branco usada apenas para separação visual.
62	COLUNAS_SAIDA = [	Inicia a lista de colunas finais exportadas.

Linha	Código	Explicação
63	"DATAVERIFICACAO", "RLD_ID", "SERVIÇO", "IDFISCAL", "FISCAL", "IDMOTORISTA", "MOTORISTA", "COD_PROPR",	Declara, em ordem, os campos que formarão a saída final, incluindo identificação, fiscalização, endereço, geocodificação, links e distâncias.
64	"NOMEFANTASIA", "NR_CGC_INDICE", "CODITEM", "DT_HORA_VERIF", "DESC_ITEM", "NR_SERIE", "NR_INMETRO",	Declara, em ordem, os campos que formarão a saída final, incluindo identificação, fiscalização, endereço, geocodificação, links e distâncias.
65	"ESPECIE", "INDICEDOC", "RESULTADO", "REGIONAL", "ENDERECO_PADRONIZADO", "ID_EQUIPE",	Declara, em ordem, os campos que formarão a saída final, incluindo identificação, fiscalização, endereço, geocodificação, links e distâncias.
66	"ID_TIPO_FISCALIZACAO", "ID_REGISTRO_FISCALIZACAO", "TOTAL_VISITAS_DIA",	Declara, em ordem, os campos que formarão a saída final, incluindo identificação, fiscalização, endereço, geocodificação, links e distâncias.
67	"LINK_ROTA_TRECHO_GOOGLE_MAPS", "LINK_RETORNO_SEDE_GOOGLE_MAPS",	Declara, em ordem, os campos que formarão a saída final, incluindo identificação, fiscalização, endereço, geocodificação, links e distâncias.
68	"BAIRRO", "MUNICIPIO", "CEP", "LATITUDE", "LONGITUDE", "PRECISAO_GEO",	Declara, em ordem, os campos que formarão a saída final, incluindo identificação, fiscalização, endereço, geocodificação, links e distâncias.
69	"DISTANCIA_DIRETA_KM", "DISTANCIA_RODOVIARIA_ESTIMADA_KM", "TEMPO_DESLOCAMENTO_MIN"]	Declara, em ordem, os campos que formarão a saída final, incluindo identificação, fiscalização, endereço, geocodificação, links e distâncias.
70	``	Linha em branco usada apenas para separação visual.

Linha	Código	Explicação
71	RE_END = re.compile(	Inicia a expressão regular para o formato antigo de endereço.
72	r"^(?P<log>.*?),\s*(?P<num>[^-]*?)\s*-\s*(?P<bairro>.*?)\s*-\s*"	Captura logradouro, número, bairro, município e CEP no padrão com hífens e marcador CEP.
73	r"(?P<mun>[^-]*?)\s*-\s*CEP:\s*(?P<cep>\d*)\s*$"	Captura logradouro, número, bairro, município e CEP no padrão com hífens e marcador CEP.
74	)	Fecha a expressão regular.
75	``	Linha em branco usada apenas para separação visual.
76	``	Linha em branco usada apenas para separação visual.
77	# ---------- UTILIDADES, GEOCODIFICAÇÃO E ROTAS ----------	Inicia a seção de utilidades, geocodificação e rotas.
78	def haversine_km(lat1, lon1, lat2, lon2):	Declara a função de distância Haversine.
79	"""Distância em linha reta (km), vetorizada; propaga NaN."""	Documenta que o cálculo retorna distância em linha reta e propaga NaN.
80	r = 6371.0	Define o raio médio da Terra em quilômetros.
81	p1, p2 = np.radians(lat1), np.radians(lat2)	Converte latitudes para radianos.
82	dp, dl = p2 - p1, np.radians(lon2 - lon1)	Calcula diferenças angulares de latitude e longitude em radianos.
83	a = np.sin(dp / 2) ** 2 + np.cos(p1) * np.cos(p2) * np.sin(dl / 2) ** 2	Calcula o componente central da fórmula de Haversine.
84	return 2 * r * np.arcsin(np.sqrt(a))	Converte o ângulo central em distância quilométrica.
85	``	Linha em branco usada apenas para separação visual.

Linha	Código	Explicação
86	``	Linha em branco usada apenas para separação visual.
87	def parse_endereco(txt: str) -> dict:	Declara a função de interpretação do endereço.
88	""""RUA X, 12 - BAIRRO - MUNICÍPIO - CEP: 12345678' -> partes.	Documenta o formato esperado, a separação segura por hífens e a normalização de CEP de sete dígitos.
89	Separa pelos hífens COM espaços (" - "), então nomes como BIRITIBA-MIRIM ficam inteiros, e o	Documenta o formato esperado, a separação segura por hífens e a normalização de CEP de sete dígitos.
90	município é sempre o último trecho antes do CEP. CEP de 7 dígitos (zero à esquerda perdido,	Documenta o formato esperado, a separação segura por hífens e a normalização de CEP de sete dígitos.
91	ex.: 8940000) é completado para 8 dígitos.""""	Documenta o formato esperado, a separação segura por hífens e a normalização de CEP de sete dígitos.
92	vazio = {"log": "", "num": "", "bairro": "", "mun": "", "cep": ""}	Cria o retorno vazio padrão quando o endereço não puder ser interpretado.
93	t = str(txt).strip()	Converte o valor para texto e remove espaços nas extremidades.
94	d = None	Inicializa o dicionário interpretado como inexistente.
95	m = re.match(r"^(.*?)\s+-\s+CEP:\s*(\d*)\s*$", t)	Tenta reconhecer o formato atual com CEP ao final.
96	if m:	Se houver correspondência, separa os blocos pelos hífens cercados por espaços.
97	partes = re.split(r"\s+-\s+", m.group(1))	Aceita apenas endereços com pelo menos logradouro, bairro e município.
98	if len(partes) >= 3:	Extrai logradouro e número e monta o dicionário com bairro, município e CEP.

Linha	Código	Explicação
99	log, _, num = partes[0].partition(",")	Extrai logradouro e número e monta o dicionário com bairro, município e CEP.
100	d = {"log": log.strip(), "num": num.strip(), "bairro": " - ".join(partes[1:-1]).strip(),	Extrai logradouro e número e monta o dicionário com bairro, município e CEP.
101	"mun": partes[-1].strip(), "cep": m.group(2)}	Extrai logradouro e número e monta o dicionário com bairro, município e CEP.
102	if d is None: # reserva: formato antigo, sem espaços ao redor do hífen	Indica o fallback para o formato antigo sem espaços ao redor dos hífens.
103	m = RE_END.match(t)	Aplica a expressão regular alternativa.
104	if not m:	Se o formato antigo também falhar, retorna o dicionário vazio.
105	return vazio	Se o formato antigo também falhar, retorna o dicionário vazio.
106	d = {k: (v or "").strip() for k, v in m.groupdict().items()}	Converte grupos capturados em valores limpos, substituindo nulos por texto vazio.
107	# "... - BIRITIBA - MIRIM" (hífen com espaços) também é o município BIRITIBA-MIRIM	Documenta um tratamento especial para BIRITIBA-MIRIM.
108	if d["mun"] == "MIRIM" and d["bairro"].endswith("BIRITIBA"):	Reconstrói o município BIRITIBA-MIRIM quando ele foi quebrado pelo hífen e ajusta o bairro.
109	d["mun"] = "BIRITIBA-MIRIM"	Reconstrói o município BIRITIBA-MIRIM quando ele foi quebrado pelo hífen e ajusta o bairro.
110	d["bairro"] = d["bairro"][:-len("BIRITIBA")].rstrip(" -")	Reconstrói o município BIRITIBA-MIRIM quando ele foi quebrado pelo hífen e ajusta o bairro.
111	n = re.match(r"\d+", d["num"]) # "738 APTO 3" -> "738"; "S/N" -> ""	Extrai apenas o primeiro número do campo, ignorando complemento e tratando S/N.

Linha	Código	Explicação
112	`    d["num"] = n.group(0) if n and n.group(0) != "0" else ""`	Mantém número apenas se ele for diferente de zero; caso contrário, deixa vazio.
113	`    if len(d["cep"]) == 7:`	Completa CEP de sete dígitos com zero à esquerda.
114	`        d["cep"] = d["cep"].zfill(8)`	Completa CEP de sete dígitos com zero à esquerda.
115	`    d["cep"] = d["cep"] if len(d["cep"]) == 8 else ""`	Aceita somente CEPs com oito dígitos.
116	`    return d`	Retorna o dicionário de partes do endereço.
117	``	Linha em branco usada apenas para separação visual.
118	``	Linha em branco usada apenas para separação visual.
119	`class Geocodificador:`	Declara a classe responsável pela consulta e cache do Nominatim.
120	`    def __init__(self, arq_cache: str):`	Declara o construtor da classe.
121	`        self.arq_cache = arq_cache`	Guarda o caminho do arquivo de cache.
122	`        self.cache = {}`	Inicializa o cache vazio.
123	`        if os.path.exists(arq_cache):`	Se o cache existir, abre o JSON e carrega as respostas anteriores.
124	`            with open(arq_cache, encoding="utf-8") as f:`	Se o cache existir, abre o JSON e carrega as respostas anteriores.
125	`                self.cache = json.load(f)`	Se o cache existir, abre o JSON e carrega as respostas anteriores.
126	`        self.sessao = requests.Session()`	Cria uma sessão HTTP reutilizável.
127	`        self.sessao.headers.update({"User-Agent": USER_AGENT})`	Configura o User-Agent da sessão.

Linha	Código	Explicação
128	`        self.novas = 0`	Inicializa o contador de consultas novas.
129	``	Linha em branco usada apenas para separação visual.
130	`    def salvar(self):`	Declara o método de persistência do cache.
131	`        with open(self.arq_cache, "w", encoding="utf-8") as f:`	Grava o cache como JSON UTF-8, preservando caracteres não ASCII.
132	`            json.dump(self.cache, f, ensure_ascii=False)`	Grava o cache como JSON UTF-8, preservando caracteres não ASCII.
133	``	Fecha o método salvar.
134	`    def consultar(self, params: dict):`	Declara o método de consulta ao Nominatim.
135	`        """Consulta o Nominatim (com cache). Retorna [lat, lon] ou None."""`	Documenta entrada, uso de cache e retorno de coordenadas.
136	`        chave = json.dumps(params, sort_keys=True, ensure_ascii=False)`	Serializa os parâmetros para formar uma chave estável e retorna imediatamente se já estiver no cache.
137	`        if chave in self.cache:`	Serializa os parâmetros para formar uma chave estável e retorna imediatamente se já estiver no cache.
138	`            return self.cache[chave]`	Continuação da instrução ou elemento sintático da lógica descrita no bloco correspondente.
139	`        base = {"format": "jsonv2", "limit": 1, "countrycodes": "br", "accept-language": "pt-BR"}`	Define parâmetros comuns: JSON, um resultado, Brasil e idioma português.
140	`        resultado = None`	Inicializa o resultado como inexistente.
141	`        for tentativa in range(4):`	Configura até quatro tentativas.

Linha	Código	Explicação
142	`        try:`	Tenta realizar a requisição HTTP com timeout de 30 segundos.
143	`            r = self.sessao.get(URL_NOMINATIM, params={**base, **params}, timeout=30)`	Tenta realizar a requisição HTTP com timeout de 30 segundos.
144	`            if r.status_code in (429, 502, 503, 504):`	Trata limites/indisponibilidade temporária com espera progressiva.
145	`                time.sleep(5 * (tentativa + 1))`	Trata limites/indisponibilidade temporária com espera progressiva.
146	`                continue`	Lança exceção para outros códigos HTTP inválidos.
147	`            r.raise_for_status()`	Interpreta o JSON e verifica se existe resultado.
148	`            dados = r.json()`	Interpreta o JSON e verifica se existe resultado.
149	`            if dados:`	Converte latitude e longitude do primeiro resultado para float.
150	`                resultado = [float(dados[0]["lat"]), float(dados[0]["lon"])]`	Converte latitude e longitude do primeiro resultado para float.
151	`            break`	Encerra as tentativas após resposta processável.
152	`        except (requests.RequestException, ValueError, KeyError):`	Captura falhas HTTP, JSON ou campos ausentes e aguarda antes de tentar de novo.
153	`            time.sleep(3 * (tentativa + 1))`	Captura falhas HTTP, JSON ou campos ausentes e aguarda antes de tentar de novo.
154	`    else:`	Se todas as tentativas falharem por rede, não grava falha no cache e permite nova tentativa em execução futura.

Linha	Código	Explicação
155	`            return None          # falha de rede: NÃO grava no cache, tenta de novo na próxima execução`	Se todas as tentativas falharem por rede, não grava falha no cache e permite nova tentativa em execução futura.
156	`        self.cache[chave] = resultado`	Armazena o resultado, inclusive None quando a busca não encontrou endereço, e incrementa consultas novas.
157	`        self.novas += 1`	Armazena o resultado, inclusive None quando a busca não encontrou endereço, e incrementa consultas novas.
158	`        time.sleep(PAUSA_S)`	Respeita a pausa mínima da API.
159	`        if self.novas % SALVAR_A_CADA == 0:`	Salva o cache periodicamente a cada 50 consultas novas.
160	`            self.salvar()`	Salva o cache periodicamente a cada 50 consultas novas.
161	`        return resultado`	Retorna a coordenada encontrada ou None.
162	``	Linha em branco usada apenas para separação visual.
163	``	Linha em branco usada apenas para separação visual.
164	`def geocodificar_endereco(g: Geocodificador, e: dict, centros: dict):`	Declara a geocodificação em cascata.
165	`    """Retorna (lat, lon, nivel). Do mais preciso ao menos preciso."""`	Documenta retorno de latitude, longitude e nível de precisão.
166	`    mun = e["mun"].title()`	Prepara município/estado/país e recupera o centro municipal conhecido.

Linha	Código	Explicação
167	`    base = {"city": mun, "state": "São Paulo", "country": "Brasil"}`	Prepara município/estado/país e recupera o centro municipal conhecido.
168	`    centro = centros.get(e["mun"])`	Prepara município/estado/país e recupera o centro municipal conhecido.
169	```	Linha em branco usada apenas para separação visual.
170	`    tentativas = []`	Inicializa a lista de tentativas.
171	`    if e["log"] and e["num"]:`	Adiciona busca pelo endereço completo quando há logradouro e número.
172	`        tentativas.append(("ENDERECO", {**base, "street": f'{e["num"]} {e["log"]}'}))`	Adiciona busca pelo endereço completo quando há logradouro e número.
173	`    if e["log"]:`	Adiciona busca somente pelo nome da rua.
174	`        tentativas.append(("RUA", {**base, "street": e["log"]}))`	Adiciona busca somente pelo nome da rua.
175	`    if e["cep"]:`	Adiciona busca pelo CEP formatado.
176	`        tentativas.append(("CEP", {"postalcode": f'{e["cep"][:5]}-{e["cep"][5:]}', "country": "Brasil"}))`	Adiciona busca pelo CEP formatado.
177	`    if e["bairro"]:`	Adiciona busca pelo bairro, município e estado.
178	`        tentativas.append(("BAIRRO", {"q": f'{e["bairro"]}, {mun}, São Paulo, Brasil'}))`	Adiciona busca pelo bairro, município e estado.
179	```	Linha em branco usada apenas para separação visual.
180	`    for nivel, params in tentativas:`	Percorre as tentativas na ordem de maior para menor precisão.
181	`        pt = g.consultar(params)`	Percorre as tentativas na ordem de maior para menor precisão.

Linha	Código	Explicação
182	`    if not pt:`	Ignora resultados vazios e continua para a próxima estratégia.
183	`        continue`	Ignora resultados vazios e continua para a próxima estratégia.
184	`    if centro and haversine_km(pt[0], pt[1], centro[0], centro[1]) > RAIO_MAX_KM:`	Descarta resultado distante mais de 40 km do centro municipal, reduzindo falsos positivos.
185	`        continue        # caiu longe demais do município: descarta`	Descarta resultado distante mais de 40 km do centro municipal, reduzindo falsos positivos.
186	`    return pt[0], pt[1], nivel`	Retorna o primeiro ponto válido com seu nível de precisão.
187	`  if centro:`	Se nenhuma busca funcionar, usa o centro do município como fallback.
188	`    return centro[0], centro[1], "MUNICIPIO"`	Se nenhuma busca funcionar, usa o centro do município como fallback.
189	`  return None, None, "NAO_ENCONTRADO"`	Se nem isso existir, retorna coordenadas nulas e nível NAO_ENCONTRADO.
190	``	Linha em branco usada apenas para separação visual.
191	``	Linha em branco usada apenas para separação visual.
192	`def geocodificar_base(df: pd.DataFrame):`	Declara a geocodificação da base.
193	`  end = pd.DataFrame({"ENDERECO_COMPLETO": df["ENDERECO_COMPLETO"].dropna().unique()})`	Obtém endereços únicos, interpreta cada endereço e junta as partes estruturadas.
194	`  partes = end["ENDERECO_COMPLETO"].apply(parse_endereco).apply(pd.Series)`	Obtém endereços únicos, interpreta cada endereço e junta as partes estruturadas.
195	`  end = pd.concat([end, partes], axis=1)`	Obtém endereços únicos, interpreta cada endereço e junta as partes estruturadas.

Linha	Código	Explicação
196	`    if LIMITE_TESTE:`	Aplica limite opcional de teste.
197	`        end = end.head(LIMITE_TESTE).copy()`	Aplica limite opcional de teste.
198	`    print(f"{len(end)} endereços únicos para geocodificar "`	Exibe quantidade de endereços e quantos ficaram sem município reconhecido.
199	`            f"({(end['mun'] == '').sum()} sem município reconhecido)")`	Exibe quantidade de endereços e quantos ficaram sem município reconhecido.
200	``	Linha em branco usada apenas para separação visual.
201	`    g = Geocodificador(ARQ_CACHE_GEO)`	Cria o geocodificador usando o cache configurado.
202	`    centros = {}`	Inicializa o dicionário de centros municipais.
203	`    for mun in sorted(m for m in end["mun"].unique() if m):`	Busca os centros dos municípios, armazena os encontrados e exibe cobertura.
204	`        pt = g.consultar({"city": mun.title(), "state": "São Paulo", "country": "Brasil"})`	Busca os centros dos municípios, armazena os encontrados e exibe cobertura.
205	`        if pt:`	Busca os centros dos municípios, armazena os encontrados e exibe cobertura.
206	`            centros[mun] = pt`	Busca os centros dos municípios, armazena os encontrados e exibe cobertura.
207	`    print(f"Centros de município localizados: {len(centros)}/{end['mun'].nunique()}")`	Busca os centros dos municípios, armazena os encontrados e exibe cobertura.
208	``	Linha em branco usada apenas para separação visual.

Linha	Código	Explicação
209	`    lat, lon, nivel = [], [], []`	Inicializa listas de latitude, longitude e nível de geocodificação.
210	`    for i, (_, linha) in enumerate(end.iterrows(), 1):`	Processa cada endereço único e acumula o resultado das coordenadas.
211	`        a, b, n = geocodificar_endereco(g, linha.to_dict(), centros)`	Processa cada endereço único e acumula o resultado das coordenadas.
212	`        lat.append(a); lon.append(b); nivel.append(n)`	Processa cada endereço único e acumula o resultado das coordenadas.
213	`        if i % 100 == 0:`	Exibe progresso a cada 100 endereços.
214	`            print(f"  {i}/{len(end)}  ({g.novas} consultas novas)")`	Exibe progresso a cada 100 endereços.
215	`    g.salvar()`	Salva o cache ao final da geocodificação.
216	`    end["LATITUDE"], end["LONGITUDE"], end["GEO_NIVEL"] = lat, lon, nivel`	Anexa latitude, longitude e nível ao dataframe de endereços.
217	`    end = end.rename(columns={"mun": "MUNICIPIO", "bairro": "BAIRRO", "cep": "CEP",`	Renomeia os campos estruturados para os nomes usados na saída.
218	`                             "log": "LOGRADOURO", "num": "NUMERO"})`	Renomeia os campos estruturados para os nomes usados na saída.
219	`    print("\nQualidade da geocodificação (endereços únicos):")`	Exibe a distribuição da qualidade/nível de geocodificação.
220	`    print(end["GEO_NIVEL"].value_counts().to_string())`	Exibe a distribuição da qualidade/nível de geocodificação.
221	`    return end`	Retorna a tabela de endereços enriquecida.
222	``	Linha em branco usada apenas para separação visual.
223	``	Linha em branco usada apenas para separação visual.

Linha	Código	Explicação
224	def obter_sede():	Declara a função que encontra a coordenada da sede e documenta o retorno.
225	"""Coordenada da sede (início/fim do dia). Retorna (lat, lon, origem)."""	Declara a função que encontra a coordenada da sede e documenta o retorno.
226	if SEDE_MANUAL:	Prioriza coordenada manual quando configurada.
227	return SEDE_MANUAL[0], SEDE_MANUAL[1], "manual"	Prioriza coordenada manual quando configurada.
228	cidade = {"city": "São José dos Campos", "state": "São Paulo", "country": "Brasil"}	Define São José dos Campos como cidade da sede.
229	consultas = [	Define consultas em cascata: endereço com CEP, grafia alternativa e CEP.
230	("endereço + CEP", {**cidade, "street": "759 Rua Sebastião Hummel", "postalcode": "12210-200"}),	Define consultas em cascata: endereço com CEP, grafia alternativa e CEP.
231	("endereço (grafia Humel)", {**cidade, "street": "759 Rua Sebastião Humel"}),	Define consultas em cascata: endereço com CEP, grafia alternativa e CEP.
232	("CEP", {"postalcode": "12210-200", "country": "Brasil"}),	Define consultas em cascata: endereço com CEP, grafia alternativa e CEP.
233	]	Define consultas em cascata: endereço com CEP, grafia alternativa e CEP.
234	for nivel, params in consultas:	Percorre as consultas e realiza requisições com tratamento local de erro.
235	try:	Percorre as consultas e realiza requisições com tratamento local de erro.
236	r = requests.get(URL_NOMINATIM, headers={"User-Agent": USER_AGENT}, timeout=15,	Consulta o Nominatim com User-Agent, formato JSON e limite de um resultado.
237	params={**params, "format": "jsonv2", "limit": 1, "countrycodes": "br"})	Consulta o Nominatim com User-Agent, formato JSON e limite de um resultado.

Linha	Código	Explicação
238	`        r.raise_for_status()`	Valida resposta e transforma o primeiro resultado em coordenadas.
239	`        dados = r.json()`	Valida resposta e transforma o primeiro resultado em coordenadas.
240	`        if dados:`	Valida resposta e transforma o primeiro resultado em coordenadas.
241	`            lat, lon = float(dados[0]["lat"]), float(dados[0]["lon"])`	Valida resposta e transforma o primeiro resultado em coordenadas.
242	`            if haversine_km(lat, lon, *SEDE_APROX) <= 15:   # sanidade: tem de ser em SJC`	Confirma que o ponto está até 15 km da aproximação de SJC antes de aceitá-lo.
243	`                return lat, lon, f"Nominatim ({nivel})"`	Confirma que o ponto está até 15 km da aproximação de SJC antes de aceitá-lo.
244	`    except (requests.RequestException, ValueError, KeyError):`	Ignora falhas esperadas e aguarda entre consultas.
245	`        pass`	Ignora falhas esperadas e aguarda entre consultas.
246	`    time.sleep(1.1)`	Ignora falhas esperadas e aguarda entre consultas.
247	`  print("AVISO: não consegui localizar a sede; usando coordenada APROXIMADA do Centro de SJC. "`	Emite aviso e usa a sede aproximada como fallback.
248	`        "Defina SEDE_MANUAL com a coordenada exata.")`	Emite aviso e usa a sede aproximada como fallback.
249	`  return SEDE_APROX[0], SEDE_APROX[1], "APROXIMADA (fallback)"`	Emite aviso e usa a sede aproximada como fallback.
250	``	Linha em branco usada apenas para separação visual.
251	``	Linha em branco usada apenas para separação visual.
252	`_ESTADO_OSRM = {"falhas_seguidas": 0, "desligado": False}`	Mantém estado global de falhas consecutivas e desligamento do OSRM.

Linha	Código	Explicação
253	``	Linha em branco usada apenas para separação visual.
254	``	Linha em branco usada apenas para separação visual.
255	def carregar_cache():	Carrega o cache de rotas se existir; caso contrário, retorna dicionário vazio.
256	__if os.path.exists(ARQ_CACHE_ROTAS):	Carrega o cache de rotas se existir; caso contrário, retorna dicionário vazio.
257	____with open(ARQ_CACHE_ROTAS, encoding="utf-8") as f:	Carrega o cache de rotas se existir; caso contrário, retorna dicionário vazio.
258	________return json.load(f)	Carrega o cache de rotas se existir; caso contrário, retorna dicionário vazio.
259	__return {}	Carrega o cache de rotas se existir; caso contrário, retorna dicionário vazio.
260	``	Linha em branco usada apenas para separação visual.
261	``	Linha em branco usada apenas para separação visual.
262	def salvar_cache(cache):	Grava o cache de rotas em JSON.
263	__with open(ARQ_CACHE_ROTAS, "w", encoding="utf-8") as f:	Grava o cache de rotas em JSON.
264	____json.dump(cache, f)	Grava o cache de rotas em JSON.
265	``	Linha em branco usada apenas para separação visual.
266	``	Linha em branco usada apenas para separação visual.
267	def rota_osrm(coords, cache):	Declara consulta de rota OSRM para uma sequência de coordenadas.

Linha	Código	Explicação
268	"""coords = [(lat, lon), ...] na ordem da rota. Retorna [[km, min], ...] por trecho, ou None."""	Documenta formato e retorno: distância e tempo por trecho.
269	chave = ";".join(f"{lon:.5f},{lat:.5f}" for lat, lon in coords)	Monta uma chave com longitude/latitude arredondadas e usa o cache quando disponível.
270	if chave in cache:	Monta uma chave com longitude/latitude arredondadas e usa o cache quando disponível.
271	return cache[chave]	Monta uma chave com longitude/latitude arredondadas e usa o cache quando disponível.
272	if _ESTADO_OSRM["desligado"]:	Desativa novas consultas após falhas consecutivas e retorna None.
273	return None	Desativa novas consultas após falhas consecutivas e retorna None.
274	legs = None	Inicializa as pernas/trechos da rota.
275	for tentativa in range(3):	Configura até três tentativas.
276	try:	Consulta OSRM em modo direção, sem geometria detalhada nem passos.
277	r = requests.get(URL_OSRM + chave, headers={"User-Agent": USER_AGENT}, timeout=30,	Consulta OSRM em modo direção, sem geometria detalhada nem passos.
278	params={"overview": "false", "steps": "false"})	Consulta OSRM em modo direção, sem geometria detalhada nem passos.
279	if r.status_code in (429, 500, 502, 503, 504):	Trata respostas transitórias com espera progressiva.
280	time.sleep(3 * (tentativa + 1))	Trata respostas transitórias com espera progressiva.
281	continue	Trata respostas transitórias com espera progressiva.

Linha	Código	Explicação
282	d = r.json()	Interpreta a resposta OK e converte metros/segundos para quilômetros/minutos.
283	if d.get("code") == "Ok":	Interpreta a resposta OK e converte metros/segundos para quilômetros/minutos.
284	legs = [[l["distance"] / 1000.0, l["duration"] / 60.0] for l in d["routes"][0]["legs"]]	Interpreta a resposta OK e converte metros/segundos para quilômetros/minutos.
285	break	Encerra tentativas após resposta processável.
286	except (requests.RequestException, ValueError, KeyError, IndexError):	Captura falhas de rede, JSON, campos ou índices e aguarda.
287	time.sleep(2 * (tentativa + 1))	Captura falhas de rede, JSON, campos ou índices e aguarda.
288	else: # falha de rede: não grava no cache, tenta de novo na próxima execução	Registra falha de rede sem cachear resultado inválido.
289	_ESTADO_OSRM["falhas_seguidas"] += 1	Registra falha de rede sem cachear resultado inválido.
290	if _ESTADO_OSRM["falhas_seguidas"] >= 5:	Após cinco falhas consecutivas, desliga OSRM e avisa que usará estimativas.
291	_ESTADO_OSRM["desligado"] = True	Após cinco falhas consecutivas, desliga OSRM e avisa que usará estimativas.
292	print("AVISO: OSRM indisponível (5 falhas seguidas); o restante usa linha reta x fator.")	Após cinco falhas consecutivas, desliga OSRM e avisa que usará estimativas.
293	return None	Retorna None quando a rota não foi obtida.
294	_ESTADO_OSRM["falhas_seguidas"] = 0	Reseta o contador de falhas, guarda a rota, aguarda e prepara retorno.
295	cache[chave] = legs	Reseta o contador de falhas, guarda a rota, aguarda e prepara retorno.

Linha	Código	Explicação
296	`    time.sleep(OSRM_PAUSA_S)`	Reseta o contador de falhas, guarda a rota, aguarda e prepara retorno.
297	`    return legs`	Retorna as pernas da rota.
298	``	Linha em branco usada apenas para separação visual.
299	``	Linha em branco usada apenas para separação visual.
300	``	Linha em branco usada apenas para separação visual.
301	``	Linha em branco usada apenas para separação visual.
302	`# ---------- PREPARAÇÃO DA BASE ----------`	Inicia a seção de preparação da base.
303	`def padronizar_endereco(s: pd.Series) -> pd.Series:`	Declara e implementa a substituição de DOUTOR por DR e AVENIDA por AV usando regex de palavra inteira.
304	`    """DOUTOR -> DR e AVENIDA -> AV (mesma padronização do ETL anterior)."""`	Declara e implementa a substituição de DOUTOR por DR e AVENIDA por AV usando regex de palavra inteira.
305	`    return s.str.replace(r"\bDOUTOR\b", "DR", regex=True).str.replace(r"\bAVENIDA\b", "AV", regex=True)`	Declara e implementa a substituição de DOUTOR por DR e AVENIDA por AV usando regex de palavra inteira.
306	``	Linha em branco usada apenas para separação visual.
307	``	Linha em branco usada apenas para separação visual.
308	`def preparar(df: pd.DataFrame) -> pd.DataFrame:`	Declara a função principal de preparação.
309	`    df = df.copy()`	Copia o dataframe para evitar alteração inesperada do original.

Linha	Código	Explicação
310	`    df["LINHA_ORIGINAL"] = np.arange(1, len(df) + 1)`	Cria a linha original, começando em 1, para rastreabilidade.
311	`    norm = df["ENDERECO_COMPLETO"].astype(str).str.replace(r"\s+", " ", regex=True).str.strip().str.upper()`	Normaliza espaços, remove espaços externos, converte para maiúsculas, preserva nulos e cria endereço padronizado.
312	`    df["ENDERECO_NORM"] = norm.where(df["ENDERECO_COMPLETO"].notna(), np.nan)`	Normaliza espaços, remove espaços externos, converte para maiúsculas, preserva nulos e cria endereço padronizado.
313	`    df["ENDERECO_PADRONIZADO"] = padronizar_endereco(df["ENDERECO_NORM"])`	Normaliza espaços, remove espaços externos, converte para maiúsculas, preserva nulos e cria endereço padronizado.
314	``	Linha em branco usada apenas para separação visual.
315	`    idf = pd.to_numeric(df["IDFISCAL"], errors="coerce").fillna(0).astype(int)`	Converte identificadores para inteiros, tratando inválidos/nulos como zero.
316	`    idm = pd.to_numeric(df["IDMOTORISTA"], errors="coerce").fillna(0).astype(int)`	Converte identificadores para inteiros, tratando inválidos/nulos como zero.
317	`    df["ID_EQUIPE"] = "EQP-" + idf.astype(str) + "-" + idm.astype(str)`	Monta ID_EQUIPE no padrão EQP-fiscal-motorista.
318	``	Linha em branco usada apenas para separação visual.
319	`    servicos = sorted(df["SERVIÇO"].dropna().unique())`	Ordena serviços, cria códigos FISC-001 etc. e mapeia o código para cada linha.
320	`    mapa = {s: f"FISC-{i:03d}" for i, s in enumerate(servicos, 1)}`	Ordena serviços, cria códigos FISC-001 etc. e mapeia o código para cada linha.
321	`    df["ID_TIPO_FISCALIZACAO"] = df["SERVIÇO"].map(mapa)`	Ordena serviços, cria códigos FISC-001 etc. e mapeia o código para cada linha.

Linha	Código	Explicação
322	`    df["ID_REGISTRO_FISCALIZACAO"] = "FISC-REC-" + df["LINHA_ORIGINAL"].astype(str).str.zfill(6)`	Cria identificador de registro com a linha original preenchida com zeros à esquerda.
323	``	Linha em branco usada apenas para separação visual.
324	`    dt = pd.to_datetime(df["DT_HORA_VERIF"], format="%d/%m/%Y %H:%M:%S", errors="coerce")`	Converte os campos de data/hora e data para datetime com os formatos esperados.
325	`    d2 = pd.to_datetime(df["DATAVERIFICACAO"], format="%d/%m/%y", errors="coerce")`	Converte os campos de data/hora e data para datetime com os formatos esperados.
326	`    df["DT"] = dt`	Cria DT e DIA; usa a data sem hora ou, em caso de falha, DATAVERIFICACAO.
327	`    df["DIA"] = dt.dt.normalize().fillna(d2)`	Cria DT e DIA; usa a data sem hora ou, em caso de falha, DATAVERIFICACAO.
328	`    return df`	Retorna a base preparada.
329	``	Linha em branco usada apenas para separação visual.
330	``	Linha em branco usada apenas para separação visual.
331	`def montar_visitas(df: pd.DataFrame) -> pd.DataFrame:`	Declara a criação de uma linha por visita, definida por equipe, dia e estabelecimento.
332	`    """Uma linha por visita: equipe + dia + estabelecimento (COD_PROPR)."""`	Declara a criação de uma linha por visita, definida por equipe, dia e estabelecimento.
333	`    return (df.groupby(["ID_EQUIPE", "DIA", "COD_PROPR"], sort=False)`	Agrupa visitas e conserva fiscal, nome, menor horário e endereço representativo de cada estabelecimento.

Linha	Código	Explicação
334	⠀⠀⠀⠀⠀⠀⠀.agg(IDFISCAL=("IDFISCAL", "first"), FISCAL=("FISCAL", "first"),	Agrupa visitas e conserva fiscal, nome, menor horário e endereço representativo de cada estabelecimento.
335	⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀HORA_INICIO=("DT", "min"),	Agrupa visitas e conserva fiscal, nome, menor horário e endereço representativo de cada estabelecimento.
336	⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀ENDERECO_NORM=("ENDERECO_NORM", "first"),	Agrupa visitas e conserva fiscal, nome, menor horário e endereço representativo de cada estabelecimento.
337	ENDERECO_PADRONIZADO=("ENDERECO_PADRONIZADO", "first"))	Agrupa visitas e conserva fiscal, nome, menor horário e endereço representativo de cada estabelecimento.
338	⠀⠀⠀⠀⠀⠀⠀.reset_index())	Agrupa visitas e conserva fiscal, nome, menor horário e endereço representativo de cada estabelecimento.
339	``	Linha em branco usada apenas para separação visual.
340	``	Linha em branco usada apenas para separação visual.
341	def descartar_fiscais(df: pd.DataFrame, vis: pd.DataFrame):	Declara o filtro de fiscais.
342	⠀⠀por_fiscal = vis.groupby(["IDFISCAL", "FISCAL"]).size().rename("VISITAS").reset_index()	Conta visitas por fiscal e relaciona a quantidade de linhas originais.
343	⠀⠀por_fiscal["LINHAS"] = por_fiscal["IDFISCAL"].map(df.groupby("IDFISCAL").size())	Conta visitas por fiscal e relaciona a quantidade de linhas originais.
344	⠀⠀por_fiscal["SITUACAO"] = np.where(por_fiscal["VISITAS"] < MIN_VISITAS_FISCAL, "DESCARTADO", "MANTIDO")	Classifica como descartado abaixo do mínimo e lista os fiscais removidos.
345	⠀⠀descartados = por_fiscal.loc[por_fiscal["SITUACAO"] == "DESCARTADO", "IDFISCAL"]	Classifica como descartado abaixo do mínimo e lista os fiscais removidos.
346	⠀⠀print(f"\nFiscais mantidos: {(por_fiscal['SITUACAO'] == 'MANTIDO').sum()} | descartados "	Imprime resumo de mantidos/descartados e tabela ordenada por visitas.

Linha	Código	Explicação
347	`        f"(menos de {MIN_VISITAS_FISCAL} visitas): {len(descartados)}")`	Imprime resumo de mantidos/descartados e tabela ordenada por visitas.
348	`    print(por_fiscal.sort_values("VISITAS", ascending=False).to_string(index=False))`	Imprime resumo de mantidos/descartados e tabela ordenada por visitas.
349	`    return df[~df["IDFISCAL"].isin(descartados)].copy(), vis[~vis["IDFISCAL"].isin(descartados)].copy(), por_fiscal`	Retorna base, visitas e tabela de auditoria somente para fiscais mantidos.
350	``	Linha em branco usada apenas para separação visual.
351	``	Linha em branco usada apenas para separação visual.
352	`# ---------------------------- TRECHOS, LINKS E DISTÂNCIAS ----------------------------`	Inicia a seção de trechos, links e distâncias.
353	`def link_maps(origem, destino):`	Declara o gerador de links e retorna vazio se origem/destino estiver ausente.
354	`    if pd.isna(origem) or pd.isna(destino):`	Declara o gerador de links e retorna vazio se origem/destino estiver ausente.
355	`        return None`	Declara o gerador de links e retorna vazio se origem/destino estiver ausente.
356	`    return ("https://www.google.com/maps/dir/?api=1&origin=" + quote(str(origem), safe="")`	Monta URL do Google Maps com origem, destino codificados e modo driving.
357	`            + "&destination=" + quote(str(destino), safe="") + "&travelmode=driving")`	Monta URL do Google Maps com origem, destino codificados e modo driving.
358	``	Linha em branco usada apenas para separação visual.
359	``	Linha em branco usada apenas para separação visual.
360	`def calcular_trechos(vis: pd.DataFrame, sede) -> pd.DataFrame:`	Declara o cálculo dos deslocamentos das visitas.

Linha	Código	Explicação
361	`    vis = vis.sort_values(["ID_EQUIPE", "DIA", "HORA_INICIO", "COD_PROPR"]).reset_index(drop=True)`	Ordena visitas por equipe/dia/hora/estabelecimento, agrupa e cria sequência e total de visitas.
362	`    g = vis.groupby(["ID_EQUIPE", "DIA"], sort=False)`	Ordena visitas por equipe/dia/hora/estabelecimento, agrupa e cria sequência e total de visitas.
363	`    vis["SEQ"] = g.cumcount() + 1`	Ordena visitas por equipe/dia/hora/estabelecimento, agrupa e cria sequência e total de visitas.
364	`    vis["N"] = g["COD_PROPR"].transform("size")`	Ordena visitas por equipe/dia/hora/estabelecimento, agrupa e cria sequência e total de visitas.
365	`    prim, ult = vis["SEQ"] == 1, vis["SEQ"] == vis["N"]`	Identifica primeira e última visita de cada equipe-dia.
366	`    lat0 = g["LATITUDE"].shift().where(~prim, sede[0])`	Define origem como visita anterior ou sede na primeira visita.
367	`    lon0 = g["LONGITUDE"].shift().where(~prim, sede[1])`	Define origem como visita anterior ou sede na primeira visita.
368	```	Linha em branco usada apenas para separação visual.
369	`    vis["DIST_DIRETA"] = haversine_km(lat0, lon0, vis["LATITUDE"], vis["LONGITUDE"])`	Calcula linha reta entre origem e destino.
370	`    vis["DIST_ROD"] = vis["DIST_DIRETA"] * FATOR_CIRCUIDADE`	Cria estimativa rodoviária pelo fator 1,30 e calcula tempo pela velocidade média.
371	`    vis["TEMPO_MIN"] = vis["DIST_ROD"] * 60 / VELOCIDADE_KMH`	Cria estimativa rodoviária pelo fator 1,30 e calcula tempo pela velocidade média.
372	`    ret_dir = haversine_km(vis["LATITUDE"], vis["LONGITUDE"], sede[0], sede[1]).where(ult)`	Calcula distância/tempo do retorno à sede apenas na última visita.
373	`    vis["RET_DIRETA"] = ret_dir`	Calcula distância/tempo do retorno à sede apenas na última visita.

Linha	Código	Explicação
374	`    vis["RET_ROD"] = ret_dir * FATOR_CIRCUIDADE`	Calcula distância/tempo do retorno à sede apenas na última visita.
375	`    vis["RET_TEMPO_MIN"] = vis["RET_ROD"] * 60 / VELOCIDADE_KMH`	Calcula distância/tempo do retorno à sede apenas na última visita.
376	`    vis["FONTE"] = "ESTIMADA"`	Marca a fonte inicial das estimativas.
377	``	Linha em branco usada apenas para separação visual.
378	`    origem_txt = g["ENDERECO_PADRONIZADO"].shift().where(~prim, SEDE_TEXTO_LINK)`	Gera links de trecho e retorno no Google Maps.
379	`    vis["LINK_ROTA"] = [link_maps(o, d) for o, d in zip(origem_txt, vis["ENDERECO_PADRONIZADO"])]`	Gera links de trecho e retorno no Google Maps.
380	`    vis["LINK_RETORNO"] = [link_maps(d, SEDE_TEXTO_LINK) if u else None`	Gera links de trecho e retorno no Google Maps.
381	`                        for d, u in zip(vis["ENDERECO_PADRONIZADO"], ult)]`	Gera links de trecho e retorno no Google Maps.
382	``	Linha em branco usada apenas para separação visual.
383	`    if USAR_OSRM:`	Executa a melhoria por OSRM quando habilitada.
384	`        cache = carregar_cache()`	Carrega cache de rotas.
385	`        rod, tmp = vis["DIST_ROD"].to_numpy(copy=True), vis["TEMPO_MIN"].to_numpy(copy=True)`	Copia arrays de estimativas, retornos e fonte para atualização eficiente.
386	`        rrod, rtmp = vis["RET_ROD"].to_numpy(copy=True), vis["RET_TEMPO_MIN"].to_numpy(copy=True)`	Copia arrays de estimativas, retornos e fonte para atualização eficiente.
387	`        fonte = vis["FONTE"].to_numpy(dtype=object, copy=True)`	Copia arrays de estimativas, retornos e fonte para atualização eficiente.
388	`        lat, lon = vis["LATITUDE"].to_numpy(), vis["LONGITUDE"].to_numpy()`	Obtém coordenadas, agrupa posições por equipe-dia e inicializa contadores.

Linha	Código	Explicação
389	`    dias = vis.groupby(["ID_EQUIPE", "DIA"], sort=False).indices`	Obtém coordenadas, agrupa posições por equipe-dia e inicializa contadores.
390	`    ok = falha = 0`	Obtém coordenadas, agrupa posições por equipe-dia e inicializa contadores.
391	`    for i, (_, pos) in enumerate(dias.items(), 1):`	Percorre equipes-dia; se faltar coordenada, conta falha e mantém estimativa.
392	`        if np.isnan(lat[pos]).any() or np.isnan(lon[pos]).any():`	Percorre equipes-dia; se faltar coordenada, conta falha e mantém estimativa.
393	`            falha += 1`	Percorre equipes-dia; se faltar coordenada, conta falha e mantém estimativa.
394	`            continue`	Percorre equipes-dia; se faltar coordenada, conta falha e mantém estimativa.
395	`        legs = rota_osrm([sede] + list(zip(lat[pos], lon[pos])) + [sede], cache)`	Solicita uma rota completa sede-visitas-sede e valida quantidade de pernas.
396	`        if not legs or len(legs) != len(pos) + 1:`	Solicita uma rota completa sede-visitas-sede e valida quantidade de pernas.
397	`            falha += 1`	Solicita uma rota completa sede-visitas-sede e valida quantidade de pernas.
398	`            continue`	Continuação da instrução ou elemento sintático da lógica descrita no bloco correspondente.
399	`        k = np.array([x[0] for x in legs]); m = np.array([x[1] for x in legs])`	Se válida, aplica distâncias/tempos OSRM aos trechos e ao retorno final.
400	`        rod[pos], tmp[pos], fonte[pos] = k[:-1], m[:-1], "OSRM"`	Se válida, aplica distâncias/tempos OSRM aos trechos e ao retorno final.
401	`        rrod[pos[-1]], rtmp[pos[-1]] = k[-1], m[-1]`	Se válida, aplica distâncias/tempos OSRM aos trechos e ao retorno final.

Linha	Código	Explicação
402	ok += 1	Conta sucesso, informa progresso a cada 50 grupos e persiste cache periodicamente.
403	if i % 50 == 0:	Conta sucesso, informa progresso a cada 50 grupos e persiste cache periodicamente.
404	print(f" rotas {i}/{len(dias)}")	Conta sucesso, informa progresso a cada 50 grupos e persiste cache periodicamente.
405	salvar_cache(cache)	Conta sucesso, informa progresso a cada 50 grupos e persiste cache periodicamente.
406	salvar_cache(cache)	Salva cache, exibe resumo e devolve colunas atualizadas com a fonte.
407	print(f"Rotas por estrada (OSRM): {ok} equipe-dias | estimadas por linha reta x fator: {falha} equipe-dias")	Salva cache, exibe resumo e devolve colunas atualizadas com a fonte.
408	vis["DIST_ROD"], vis["TEMPO_MIN"], vis["RET_ROD"], vis["RET_TEMPO_MIN"], vis["FONTE"] = rod, tmp, rrod, rtmp, fonte	Salva cache, exibe resumo e devolve colunas atualizadas com a fonte.
409	return vis	Retorna visitas enriquecidas.
410	``	Linha em branco usada apenas para separação visual.
411	``	Linha em branco usada apenas para separação visual.
412	# ---------- EXPORTAÇÃO ----------	Inicia a seção de exportação.
413	def limpar_texto(df: pd.DataFrame) -> pd.DataFrame:	Declara limpeza de texto e importa a regex do openpyxl para caracteres ilegais.
414	"""Remove caracteres de controle que o Excel não aceita."""	Declara limpeza de texto e importa a regex do openpyxl para caracteres ilegais.

Linha	Código	Explicação
415	`    from openpyxl.cell.cell import ILLEGAL_CHARACTERS_RE`	Declara limpeza de texto e importa a regex do openpyxl para caracteres ilegais.
416	`    df = df.copy()`	Copia a base e remove caracteres de controle das colunas textuais, retornando o resultado.
417	`    for c in df.columns:`	Copia a base e remove caracteres de controle das colunas textuais, retornando o resultado.
418	`        if pd.api.types.is_object_dtype(df[c]) or pd.api.types.is_string_dtype(df[c]):`	Copia a base e remove caracteres de controle das colunas textuais, retornando o resultado.
419	`            df[c] = df[c].map(lambda x: ILLEGAL_CHARACTERS_RE.sub("", x) if isinstance(x, str) else x)`	Copia a base e remove caracteres de controle das colunas textuais, retornando o resultado.
420	`    return df`	Copia a base e remove caracteres de controle das colunas textuais, retornando o resultado.
421	```	Linha em branco usada apenas para separação visual.
422	```	Linha em branco usada apenas para separação visual.
423	`def exportar(saida: pd.DataFrame):`	Declara a exportação dos resultados.
424	`    caminho = os.path.join(PASTA, f"{NOME_SAIDA}.xlsx")`	Monta o caminho do arquivo XLSX.
425	`    with pd.ExcelWriter(caminho, engine="openpyxl") as w:`	Cria o Excel com openpyxl e grava uma aba sem índice.
426	`        limpar_texto(saida).to_excel(w, sheet_name="Base_Geocodificada", index=False)`	Cria o Excel com openpyxl e grava uma aba sem índice.
427	`        ws = w.sheets["Base_Geocodificada"]`	Obtém a planilha e importa estilos de fonte/preenchimento.

Linha	Código	Explicação
428	`    from openpyxl.styles import Font, PatternFill`	Obtém a planilha e importa estilos de fonte/preenchimento.
429	`    for c in ws[1]:`	Formata o cabeçalho com negrito, texto branco e fundo azul.
430	`        c.font = Font(bold=True, color="FFFFFF")`	Formata o cabeçalho com negrito, texto branco e fundo azul.
431	`        c.fill = PatternFill("solid", fgColor="2E6F9E")`	Formata o cabeçalho com negrito, texto branco e fundo azul.
432	`    ws.freeze_panes = "A2"`	Congela a primeira linha e habilita filtro automático em toda a dimensão usada.
433	`    ws.auto_filter.ref = ws.dimensions`	Congela a primeira linha e habilita filtro automático em toda a dimensão usada.
434	`    for col in ws.columns:`	Ajusta largura de colunas, reservando mais espaço para links e limitando dimensões.
435	`        nome = str(col[0].value)`	Ajusta largura de colunas, reservando mais espaço para links e limitando dimensões.
436	`        largura = 28 if nome.startswith("LINK_") else min(max(len(nome) + 2, 12), 45)`	Ajusta largura de colunas, reservando mais espaço para links e limitando dimensões.
437	`        ws.column_dimensions[col[0].column_letter].width = largura`	Ajusta largura de colunas, reservando mais espaço para links e limitando dimensões.
438	`  print(f"[OK] {NOME_SAIDA}.xlsx ({len(saida)} linhas x {saida.shape[1]} colunas, 1 aba)")`	Imprime resumo do Excel gerado.
439	`  if EXPORTAR_CSV:`	Se habilitado, grava CSV UTF-8 com BOM e confirma a saída.
440	`    saida.to_csv(os.path.join(PASTA, f"{NOME_SAIDA}.csv"), index=False, encoding="utf-8-sig")`	Se habilitado, grava CSV UTF-8 com BOM e confirma a saída.
441	`    print(f"[OK] {NOME_SAIDA}.csv")`	Se habilitado, grava CSV UTF-8 com BOM e confirma a saída.

Linha	Código	Explicação
442	``	Linha em branco usada apenas para separação visual.
443	``	Linha em branco usada apenas para separação visual.
444	# ---------- PIPELINE ----------	Inicia a seção do pipeline principal.
445	def main():	Declara a função main.
446	try:	Tenta montar o Google Drive no Colab; fora do Colab, ignora a ausência do módulo.
447	from google.colab import drive	Tenta montar o Google Drive no Colab; fora do Colab, ignora a ausência do módulo.
448	drive.mount("/content/drive")	Tenta montar o Google Drive no Colab; fora do Colab, ignora a ausência do módulo.
449	except ImportError:	Tenta montar o Google Drive no Colab; fora do Colab, ignora a ausência do módulo.
450	pass	Tenta montar o Google Drive no Colab; fora do Colab, ignora a ausência do módulo.
451	``	Linha em branco usada apenas para separação visual.
452	print("== Lendo", os.path.basename(ARQ_ENTRADA))	Lê o CSV em latin1 e informa dimensões da base bruta.
453	bruto = pd.read_csv(ARQ_ENTRADA, encoding="latin1")	Lê o CSV em latin1 e informa dimensões da base bruta.
454	print(f"Base carregada: {bruto.shape[0]} linhas x {bruto.shape[1]} colunas")	Lê o CSV em latin1 e informa dimensões da base bruta.

Linha	Código	Explicação
455	`    df = preparar(bruto)`	Prepara a base, monta visitas e imprime quantidade de visitas.
456	`    vis = montar_visitas(df)`	Prepara a base, monta visitas e imprime quantidade de visitas.
457	`    print(f"Visitas (equipe + dia + estabelecimento): {len(vis)}")`	Prepara a base, monta visitas e imprime quantidade de visitas.
458	`    df, vis, por_fiscal = descartar_fiscais(df, vis)`	Descarta fiscais abaixo do mínimo e informa linhas mantidas.
459	`    print(f"Linhas mantidas: {len(df)} de {len(bruto)}")`	Descarta fiscais abaixo do mínimo e informa linhas mantidas.
460	``	Linha em branco usada apenas para separação visual.
461	`    # TOTAL_VISITAS_DIA = estabelecimentos distintos da equipe no dia`	Calcula visitas por equipe-dia e junta TOTAL_VISITAS_DIA à base detalhada.
462	`    total = vis.groupby(["ID_EQUIPE", "DIA"]).size().rename("TOTAL_VISITAS_DIA").reset_index()`	Calcula visitas por equipe-dia e junta TOTAL_VISITAS_DIA à base detalhada.
463	`    df = df.merge(total, on=["ID_EQUIPE", "DIA"], how="left")`	Calcula visitas por equipe-dia e junta TOTAL_VISITAS_DIA à base detalhada.
464	``	Linha em branco usada apenas para separação visual.
465	`    print("\n== Geocodificação")`	Inicia etapa de geocodificação.
466	`    end = geocodificar_base(pd.DataFrame({"ENDERECO_COMPLETO": df["ENDERECO_NORM"]}))`	Geocodifica endereços únicos, renomeia precisão, seleciona atributos e faz merges na base detalhada e nas visitas.

Linha	Código	Explicação
467	`    end = end.rename(columns={"ENDERECO_COMPLETO": "ENDERECO_NORM", "GEO_NIVEL": "PRECISAO_GEO"})`	Geocodifica endereços únicos, renomeia precisão, seleciona atributos e faz merges na base detalhada e nas visitas.
468	`    geo = end[["ENDERECO_NORM", "BAIRRO", "MUNICIPIO", "CEP", "LATITUDE", "LONGITUDE", "PRECISAO_GEO"]]`	Geocodifica endereços únicos, renomeia precisão, seleciona atributos e faz merges na base detalhada e nas visitas.
469	`    df = df.merge(geo, on="ENDERECO_NORM", how="left")`	Geocodifica endereços únicos, renomeia precisão, seleciona atributos e faz merges na base detalhada e nas visitas.
470	`    vis = vis.merge(geo[["ENDERECO_NORM", "LATITUDE", "LONGITUDE"]], on="ENDERECO_NORM", how="left")`	Geocodifica endereços únicos, renomeia precisão, seleciona atributos e faz merges na base detalhada e nas visitas.
471	`    if df["LATITUDE"].isna().all():`	Interrompe com erro se nenhuma coordenada tiver sido obtida.
472	`        raise RuntimeError("Nenhuma coordenada obtida: verifique a conexão e o USER_AGENT.")`	Interrompe com erro se nenhuma coordenada tiver sido obtida.
473	```	Linha em branco usada apenas para separação visual.
474	`    print("\n== Trechos, links e distâncias")`	Inicia etapa de trechos, links e distâncias.
475	`    sede_info = obter_sede()`	Obtém a sede, separa coordenadas e imprime origem utilizada.
476	`    sede = (sede_info[0], sede_info[1])`	Obtém a sede, separa coordenadas e imprime origem utilizada.
477	`    print(f"Central: {sede[0]:.6f}, {sede[1]:.6f}  [{sede_info[2]}]")`	Obtém a sede, separa coordenadas e imprime origem utilizada.
478	`    vis = calcular_trechos(vis, sede)`	Calcula os trechos das visitas.

Linha	Código	Explicação
479	__cols_v = ["ID_EQUIPE", "DIA", "COD_PROPR", "LINK_ROTA", "LINK_RETORNO", "DIST_DIRETA", "DIST_ROD", "TEMPO_MIN",	Seleciona campos de logística e junta os resultados na base original.
480	________"RET_DIRETA", "RET_ROD", "RET_TEMPO_MIN"]	Seleciona campos de logística e junta os resultados na base original.
481	__df = df.merge(vis[cols_v], on=["ID_EQUIPE", "DIA", "COD_PROPR"], how="left")	Seleciona campos de logística e junta os resultados na base original.
482	__df = df.rename(columns={"LINK_ROTA": "LINK_ROTA_TRECHO_GOOGLE_MAPS", "LINK_RETORNO": "LINK_RETORNO_SEDE_GOOGLE_MAPS",	Renomeia campos técnicos para nomes finais em português.
483	____________"DIST_DIRETA": "DISTANCIA_DIRETA_KM", "DIST_ROD": "DISTANCIA_RODOVIARIA_ESTIMADA_KM",	Renomeia campos técnicos para nomes finais em português.
484	____________"TEMPO_MIN": "TEMPO_DESLOCAMENTO_MIN"})	Renomeia campos técnicos para nomes finais em português.
485	__colunas = list(COLUNAS_SAIDA)	Monta a lista de colunas; opcionalmente inclui distância e tempo explícitos do retorno.
486	__if INCLUIR_COLUNAS_RETORNO:	Monta a lista de colunas; opcionalmente inclui distância e tempo explícitos do retorno.
487	____df = df.rename(columns={"RET_ROD": "DISTANCIA_RETORNO_SEDE_KM", "RET_TEMPO_MIN": "TEMPO_RETORNO_SEDE_MIN"})	Monta a lista de colunas; opcionalmente inclui distância e tempo explícitos do retorno.
488	____colunas += ["DISTANCIA_RETORNO_SEDE_KM", "TEMPO_RETORNO_SEDE_MIN"]	Monta a lista de colunas; opcionalmente inclui distância e tempo explícitos do retorno.
489	``	Linha em branco usada apenas para separação visual.
490	__saida = df.sort_values("LINHA_ORIGINAL")[colunas].reset_index(drop=True)	Ordena pela linha original, seleciona colunas finais e reinicia índice.

Linha	Código	Explicação
491	`    for c in ("IDMOTORISTA", "NR_INMETRO", "IDFISCAL", "TOTAL_VISITAS_DIA"):`	Converte identificadores e total de visitas para inteiros anuláveis do pandas.
492	`        saida[c] = pd.to_numeric(saida[c], errors="coerce").astype("Int64")`	Converte identificadores e total de visitas para inteiros anuláveis do pandas.
493	`    saida["CEP"] = saida["CEP"].astype("string")`	Converte CEP para string, preservando zeros à esquerda.
494	`    for c in ("LATITUDE", "LONGITUDE"):`	Arredonda coordenadas para seis casas.
495	`        saida[c] = saida[c].round(6)`	Arredonda coordenadas para seis casas.
496	`    for c in ("DISTANCIA_DIRETA_KM", "DISTANCIA_RODOVIARIA_ESTIMADA_KM"):`	Arredonda distâncias para três casas.
497	`        saida[c] = saida[c].round(3)`	Arredonda distâncias para três casas.
498	`    saida["TEMPO_DESLOCAMENTO_MIN"] = saida["TEMPO_DESLOCAMENTO_MIN"].round(2)`	Arredonda tempo para duas casas.
499	`    globals().update(df=df, vis=vis, saida=saida, por_fiscal=por_fiscal)`	Expõe dataframes no escopo global, útil para inspeção no notebook.
500	```	Linha em branco usada apenas para separação visual.
501	`    print("\n== Exportação")`	Inicia e executa a exportação.
502	`    exportar(saida)`	Inicia e executa a exportação.
503	```	Linha em branco usada apenas para separação visual.
504	`    print("\n=== Precisão da geocodificação (linhas) ===")`	Exibe distribuição final da precisão de geocodificação por linha.
505	`print(saida["PRECISAO_GEO"].value_counts(dropna=False).to_string())`	Exibe distribuição final da precisão de geocodificação por linha.

Linha	Código	Explicação
506	`    print(f"\nFiscais: {saida['FISCAL'].nunique()} | municípios: {saida['MUNICIPIO'].nunique()} | "`	Exibe quantidade de fiscais, municípios e grupos equipe-dia.
507	`        f"equipes-dia: {saida.groupby(['ID_EQUIPE', 'DATAVERIFICACAO']).ngroups}")`	Exibe quantidade de fiscais, municípios e grupos equipe-dia.
508	`    print("Concluído.")`	Informa conclusão do processo.
509	``	Linha em branco usada apenas para separação visual.
510	``	Linha em branco usada apenas para separação visual.
511	`if __name__ == "__main__":`	Executa main somente quando o arquivo é rodado diretamente, evitando execução automática em importações.
512	`    main()`	Executa main somente quando o arquivo é rodado diretamente, evitando execução automática em importações.

7. Conclusão

O programa está estruturado em etapas bem separadas e possui mecanismos importantes de rastreabilidade, cache, retentativa e fallback. O resultado é adequado para alimentar um dashboard de acompanhamento de fiscalizações e deslocamentos. Para o objetivo de visualizar semanas filtradas por mês no Power BI, a principal complementação será uma dimensão calendário com data, mês/ano, número da semana e chave de ordenação, relacionada à data da base gerada.